

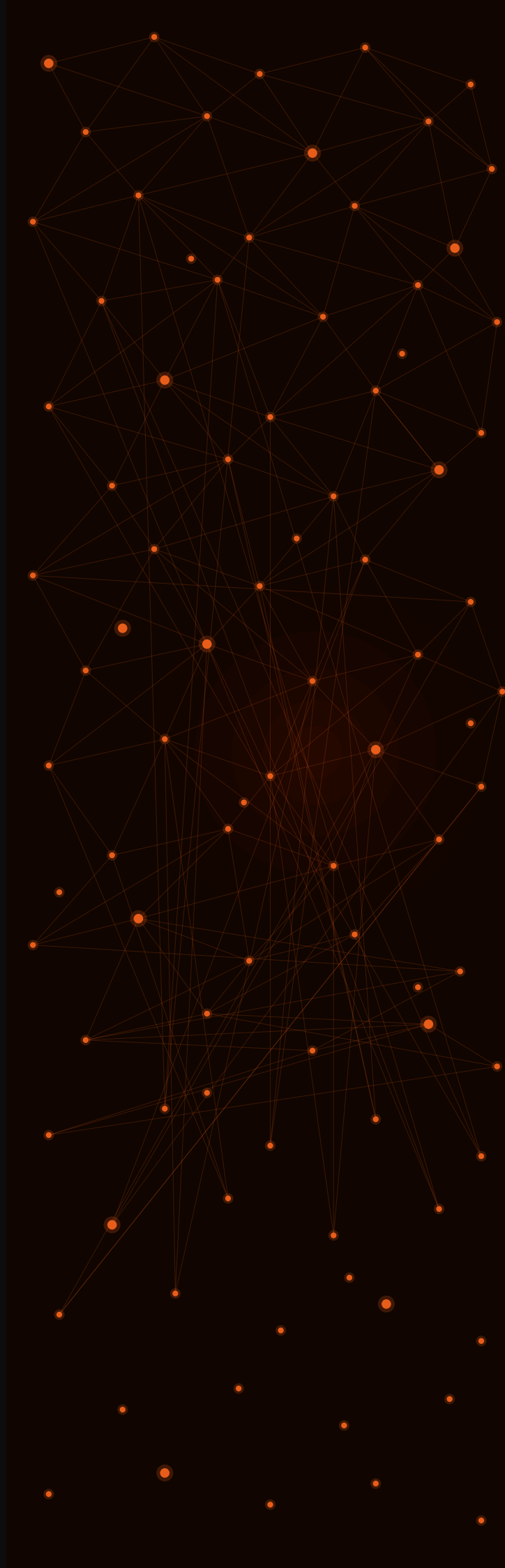


Empowering teams to deliver software better

Hive Mind Agentic Orchestration

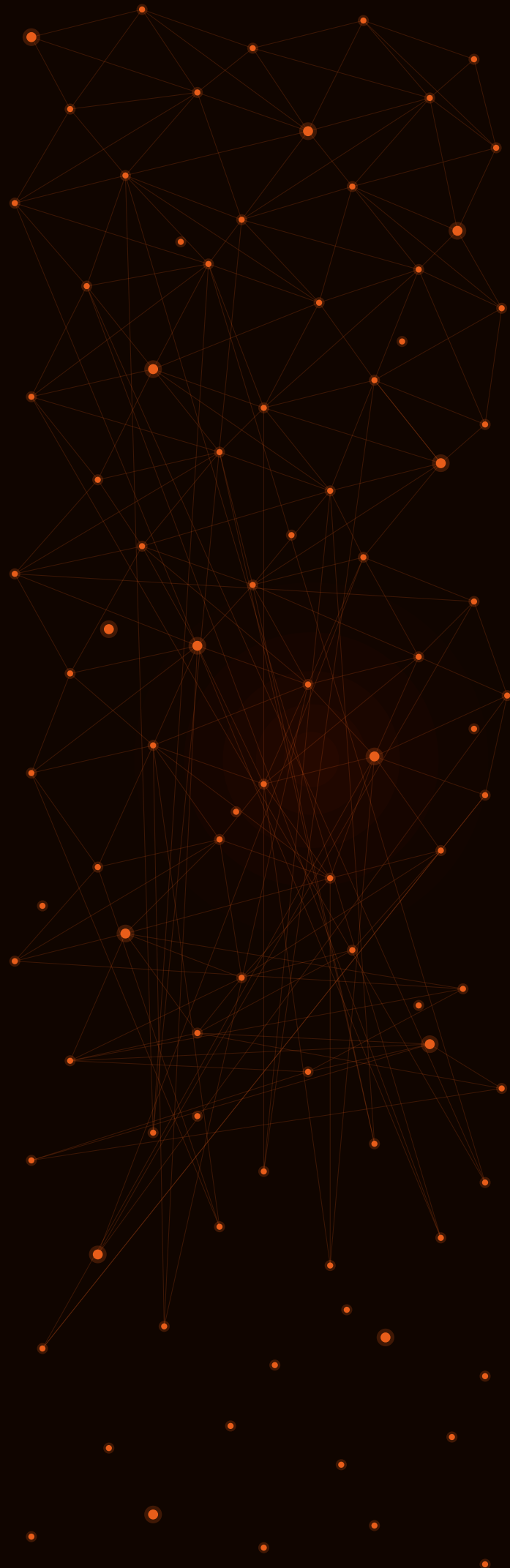
Framework and reference

April 2026



Part 1

Orchestration framework



Orchestration framework — Contents

Solving The Coordination Problem	3
Scope	4
Design Principles	5
The Layer Model	7
Execution Model	8
Source of Truth Hierarchy	9
The Rules-Standards Split	10
The Agent Roster	11
Workflow Architecture	12
Orchestration Patterns	13
Recovery Design	14
Templates and Validators	15
Limitations	16
Further Reading	17

Solving The Coordination Problem

Getting an LLM to write a function is table stakes in 2026. Getting eighteen of them to build a system together without losing their context is not.

Without deliberate coordination, multi-agent workflows break in familiar ways. Agents do not know what the previous agent did so risk redoing work that has already been done. And it's done slightly differently, because each agent makes slightly different judgements. Reviewers lack a consistent standard to check against. Testers write tests against the code as-built rather than the intended interface, ossifying bugs into the test suite. Sessions compact mid-sprint, systemising forgetfulness. The symptoms look unpredictable, a different thing going wrong each time. But the outcome is the same: wasted effort, no consistency, incorrect assumptions, and no mechanism to prevent any of it. These are coordination problems, and they require coordination solutions.

This document describes a framework designed to solve them. It is structured as a portable `context/` directory that encodes who does work, how work should be done, when work happens, what gets produced, and how quality is maintained. The patterns it applies (specialisation, explicit orchestration, durable context, verification over assertion) draw on established practices in distributed systems engineering and software delivery, adapted here for multi-agent coordination.

Scope

This is an **orchestration framework**, not an application architecture. It does not prescribe how to structure FastAPI routes or React components. It prescribes how AI agents should coordinate whilst building them.

It answers five questions:

- ▶ **Who** does work? Specialised agent definitions.
 - ▶ **How** should work be done? Standards and rules.
 - ▶ **When** does work happen? Workflow phase definitions.
 - ▶ **What** gets produced? Templates and output conventions.
 - ▶ **How is quality maintained?** Review gates, tests, and validators.
-

Design Principles

Three strategic ideas shape every structural decision: if it matters, it should be code (everything as code); humans govern the verification systems rather than reviewing every line (humans above the loop); and the binding constraint keeps moving, so the framework must move with it (elevate the next constraint). The principles below are the framework's specific expression of those ideas.

Everything important is code. Specifications, rules, agent definitions, workflows, templates, and handoffs are all versioned artefacts in the repository. The `context/` directory is the codified operating model. If knowledge is not committed and governed, it drifts, and drifted context degrades agent output more reliably than a weak model does.

Context durability has higher leverage than code quality. Code quality is downstream of context quality. An agent that receives a clear handoff, a focused task, and the correct rules will produce better code than a capable agent operating blind. The critical path is usually context plus workflow, not model capability; a better model with poor context loses to a weaker model with strong context. The framework treats documentation, tasks, and handoffs as first-class deliverables, not optional support material.

Specialist agents outperform generalists at the cost of coordination overhead. A single large prompt that covers all responsibilities tends toward diluted output. Eighteen focused agents, each with a bounded responsibility, a curated rule set, and a defined workflow position, produce more reliable work and cleaner handoffs. The trade-off is real: 18 agents means 18 handoff boundaries and 18 prompt injections. For production-quality work, the improved output quality and accountability justify this cost.

The orchestrator controls the blast radius. The orchestrator selects the agent, constrains the files it may touch, injects only the relevant rules and standards, monitors progress, and sequences the commits. No agent has unconstrained access to the repository. This is why the framework defaults to a hive model rather than a swarm: specialist agents work within bounded scopes under centralised coordination, rather than exploring the problem space independently. The orchestrator carries the coordination cost so that individual agents remain focused and their impact remains contained.

Coordination must be explicit. Agents need context, not instructions. They do not attend stand-ups, read between the lines, or ask clarifying questions mid-task. Implicit coordination (relying on agents to infer the correct sequence, ownership boundaries, or quality bar) is where hidden coupling and inconsistent behaviour originate. The framework encodes dependencies, gates, file ownership, and parallel execution boundaries in workflow YAML. Where context is ambiguous, agent rules encourage escalation rather than assumption. What is not in the file is not in the process.

Speed and rigour are sequenced, not opposed. The New DevX workflow sequences four phases (Prototype, Specify, Build, Deploy), each with a different standard of rigour matched to its purpose. Not all work needs 18 agents and mandatory quality gates. `prototype.yaml` provides fast iteration with minimal agents and no gates, because exploration matters more than engineering discipline when prototyping. `build.yaml` provides the full TDD cycle with parallel specialists, review sequencing, and coverage gates, because production quality is non-negotiable for deployed code. The workflow determines the rigour.

Humans govern above the loop. TDD phases, coverage gates, browser screenshots, review agents, and pre-commit validators bias the framework toward outputs that can be checked rather than merely asserted. An agent reporting "all tests pass" is not evidence. A command's stdout showing green is. Human attention moves from reviewing every line to designing the verification systems, classifying risk, and intervening where automated checks are insufficient.

The constraint keeps moving. Accelerating code generation does not eliminate bottlenecks; it relocates them. The binding constraint will migrate from code to review, from review to deployment, from deployment to intent clarity. The operating model must follow the constraint, not defend a fixed process.

The `continuous-improvement` workflow is the mechanism for this: in incident mode, failures are traced through a causal chain (symptom, mechanism, file, gap) to a fixable defect in the framework itself; in retrospective mode, commit telemetry, interruption logs, and incident patterns are parsed to surface recurring weaknesses in rules, standards, or templates. Both modes target the coordination and verification systems, not just the code those systems govern.

Portability is a structural requirement. Agent definitions are markdown files. Workflows are YAML. The framework includes adapter guidance for LangGraph, CrewAI, AutoGen, Cursor, and other runtimes. The current implementation leans on Claude Code, and adapting to a different runtime requires real work. The design ensures that no single tool is a hard dependency. See `context/docs/agentic-framework-reference.md` (Portability and Framework Adapters) for cross-runtime integration guidance.

The Layer Model

The framework is organised as a context hierarchy: a layered directory structure ordered below by how frequently each layer is accessed during execution. The layers that touch every agent spawn sit at the top; the layers consulted once per task or occasionally sit at the bottom.

Layer	Location	Responsibility	Access frequency
Rules	context/rules/	State what must happen: short, binary, injectable constraints	Every agent spawn: glob-matched and injected into the prompt
Agents	context/agents/	Define who: specialised roles with frontmatter linking rules and standards	Every agent spawn: orchestrator reads the definition to resolve rules, standards, and file scope
Templates	context/templates/	Define what shape: standardised formats for reviews, tasks, handoffs, design docs	Every output: agents consult templates when producing artefacts
Standards	context/standards/	Explain why and how: engineering rationale for coding, testing, security, documentation	On demand: agents load a standard when deeper reference is needed
Scripts	context/scripts/	Enforce everything above: validators, generators, git hooks	Every commit: pre-commit validators check output against rules
Workflows	context/workflows/	Define when: phase order, dependencies, gates, outputs, recovery	Once per task: orchestrator reads at the start to plan the work
Docs	context/docs/	Provide further reading: design rationale, strategic context, orchestration patterns, workflow guides, portability how-tos	Human reference: guides and design documents for contributors and operators, not consumed by agents
Personas	context/persona/	Provide voice: writing style for human-facing content only	Content workflows only: applied when writing human-facing material

The layering determines where changes should be made. To change what agents do, edit a workflow. To change how they do it, edit a standard or rule. To change who does it, edit an agent definition. To change what the output looks like, edit a template. Nothing bleeds across boundaries unless explicitly designed to.

Execution Model

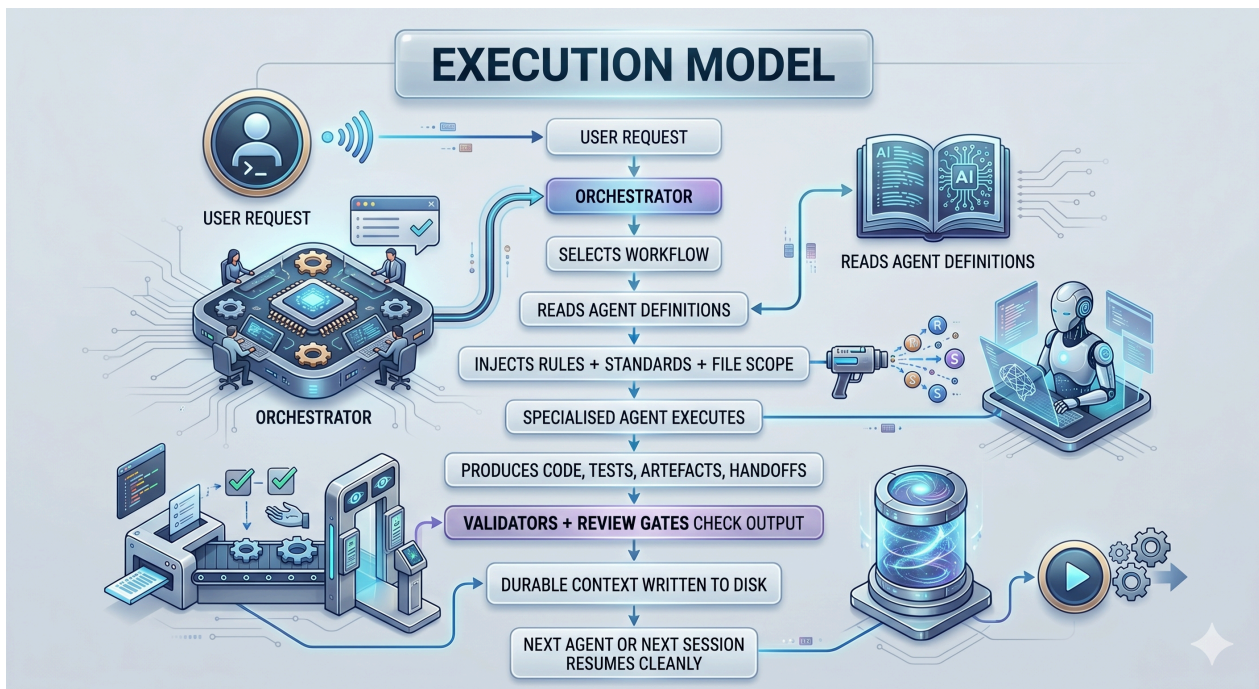


Figure 1: Execution Model

1. The **orchestrator** selects a workflow based on the task type (e.g. `build.yaml` for feature work, `bugfix.yaml` for defect resolution).
2. The workflow defines the phase sequence: plan, then write failing tests, then implement, then refactor, then review.
3. For each phase, the orchestrator reads the agent definition, resolves which rules apply by glob-matching the task's files against each rule's scope, and injects them into the spawn prompt.
4. The agent executes within its assigned file scope. Agents do not commit; only the orchestrator commits, one at a time, preventing ref-lock collisions when parallel agents finish simultaneously.
5. Validators and review gates check the output against the workflow's validation criteria.
6. Handoffs, task updates, and generated registries record what happened, ensuring the next agent or next session can resume without reconstruction.

The core design decision: orchestration state lives on disk, not in a conversation. Conversations are ephemeral. Disk is durable.

Source of Truth Hierarchy

Not all framework files carry equal authority. Some define behaviour; others project or summarise it. Editing the wrong tier is a common contributor mistake.

Tier	Examples	Action
Authoritative	context/workflows/ *.yaml, context/agents/ *.md, context/rules/ *.mdc, context/ standards/*.md	Edit these to change framework behaviour
Derived	AGENTS.md, CLAUDE.md	Read for orientation; never hand-edit. Regenerate from source.
Operator guidance	context/docs/*.md, README.md, this document	Use to understand and apply the framework
Enforcement	context/scripts/ validators/*, context/ scripts/prepare- commit-msg.*	Use to keep the repository aligned with the design

The principle: change the authoritative source, let derived projections follow.

The Rules-Standards Split

This is the most consequential design decision in the framework.

Rules are under 200 tokens each. Examples: “use `uv run pytest`, not bare `pytest`”; “agents do not commit; the orchestrator commits on their behalf.” They are designed for repeated injection into spawn prompts without significant context window cost.

Standards run to hundreds of lines. They explain why the framework uses `uv`, how TDD phases relate to each other, what constitutes a good handoff, and when to escalate versus assume. They are reference material, consulted on demand.

The split is driven by the operating triangle: execution time, token cost, and autonomy horizon. Specifically the token cost axis. A typical agent spawn injects 5-8 rules at ~100-200 tokens each, plus the agent definition at ~500-1,000 tokens, totalling roughly 1,500-2,500 tokens of framework overhead. Loading a single full standard would add 2,000-5,000 tokens, doubling or tripling the injection cost for material the agent may not need. The split keeps the execution path lean and the reference path comprehensive.

An agent executing a simple Python task needs the 15-token rule that says “run tests with `uv run pytest`.” A reviewer evaluating whether the tests are sufficient needs the full testing standard. The framework serves both without forcing either to carry the other’s weight.

The Agent Roster

The framework defines 18 specialised agents, organised by kind of judgement rather than technology alone:

- ▶ **Discovery:** product-expert (elicits requirements), product-owner (formalises them)
- ▶ **Design:** solution-architect, database-designer, api-designer, ui-designer, visual-designer
- ▶ **Implementation:** python-coder, typescript-coder
- ▶ **Testing:** functional-tester (TDD), ui-tester (browser automation)
- ▶ **Review:** tech-lead (the gate), code-reviewer, principles-reviewer, security-tester
- ▶ **Operations:** devops, documentation, workflow-analyst
- ▶ **Coordination:** orchestrator

Each agent's frontmatter declares which rules and standards it requires. The orchestrator resolves the applicable rules by matching file globs against the task scope, injecting only what is relevant.

This produces narrow, focused prompts with clear handoff boundaries. The roster is extensible: copy the agent template, fill in the frontmatter, add the agent to a workflow phase, and regenerate.

AGENTS.md is generated, not authored. A generator script (`context/scripts/generators/generate_agents_md.py`) reads every workflow YAML and every agent definition, then produces `AGENTS.md` as a single derived file containing the phase sequence, agent dispatch rules, spawn patterns, and state recovery procedures. This is the framework's portability mechanism: the same authoritative sources (workflow YAML and agent definitions) can be projected into `AGENTS.md` for one runtime, `CLAUDE.md` for another, or any format a different framework requires. The generator is a thin adapter; the sources are the portable asset. The generator is also the enforcement point for consistency. If an agent definition references a rule that does not exist, or a workflow references an agent that has no definition, the mismatch surfaces at generation time rather than at runtime. Editing `AGENTS.md` by hand is always wrong; editing the sources and regenerating is the only valid path.

Workflow Architecture

Workflows are YAML files encoding phase dependencies, agent assignments, quality gates, validation criteria, state recovery procedures, and execution rules. The framework ships nine:

Workflow	Purpose
design	Discovery through design review; run before build
build	Primary TDD loop: plan, red, green, blue, review, regress, docs cleanup
prototype	Fast iteration, no quality gates; rewrite before production
deploy	GCP cloud deployment after local verification
bugfix	Empirical reproduction, diagnosis, fix, verification
full-test	Complete regression across all modules
content	Human-facing writing with personas
continuous-improvement	Incident response and retrospective analysis
retrospective	Process review and pattern identification

Per-file summaries, phase tables, and how **design → build → deploy** compose as default delivery: `context/docs/agentik-framework-reference.md` (Workflows).

Storing orchestration logic in versionable YAML rather than ad hoc conversation makes it inspectable, diffable, reviewable, and recoverable after context compaction. The orchestration process itself is a governed, versioned artefact. The trade-off is reduced flexibility for novel situations. The `prototype.yaml` workflow addresses this by providing a minimal path with no gates or rigid sequencing, suited to exploratory work where speed matters more than rigour. For production-quality work, inspectable process consistently outperforms adaptive improvisation.

Orchestration Patterns

The framework documents five coordination patterns, from simple to complex:

1. **Single Agent:** one specialist, one task, no dependencies
2. **Sequential Chain:** agents in strict order (A then B then C)
3. **Hive:** parallel agents sharing artefacts (tests, API specs, plans) with coordination points before and after — this is what `parallel: true` means in shipped workflow YAML
4. **Iterative Loop:** review-fix cycles until approval

Parallel swarm (independent agents exploring alternative solutions in parallel) is **not** encoded in `context/workflows/*.yaml`; it appears only as forward-looking product narrative in `context/docs/vision.md` and under **Limitations** below.

Production workflows combine these. **Design** adds one parallel phase (database and API designers on the same architecture handoff). **Build** uses a sequential chain for planning, a **hive** for parallel Python and TypeScript implementation against the **same** test contract (with disjoint file scopes), parallel **review** passes that still share one plan or codebase, and iterative loops at quality gates. Most `parallel: true` phases in the shipped YAML are hive-style coordination, not independent “swarms.” The hive pattern is the hardest to manage well and where explicit orchestration adds the most value. The framework’s title reflects this emphasis.

Recovery Design

Context compaction during active work is a routine occurrence in long-running agent sessions: all accumulated state vanishes and the next agent starts from zero. The framework treats this systemised forgetfulness as a first-class design concern rather than an edge case.

Every workflow YAML includes a `state_recovery` section listing the files an orchestrator should read to reconstruct its position. Every agent handoff writes durable state to disk. The `HANDOFF.md` file records the active phase before agents are spawned, not after they report back. This ensures the phase survives even if compaction occurs during execution.

Agent session telemetry is recorded in git commits via a `prepare-commit-msg` hook. Token usage, duration, and interaction count are all reconstructable from `git log`, eliminating the need for a separate metrics database.

The overhead is real: pre-spawn handoff writes, `state_recovery` blocks in every workflow, telemetry in every commit. The justification is straightforward: a single failed recovery (agents re-reading entire codebases, duplicating completed work, losing review context) wastes more tokens than the cumulative cost of pre-emptive state recording. Durable context is also the primary defence against the comprehension gap, where agent throughput outpaces the organisation's ability to understand what changed. If the trail of decisions is inspectable, both agents and humans can reconstruct what happened without relying on conversation memory.

Templates and Validators

The framework ships 15 output templates and 7 pre-commit validators.

Templates standardise recurring artefacts: requirements, tasks, bugs, reviews, handoffs, design docs, commit messages, PR descriptions, and agent definitions. Standardised shapes reduce cognitive load for both producing and consuming agents, and make downstream automation reliable.

Validators enforce rules at commit time. A rule without a validator is a suggestion that we trust agents will follow. A rule with a validator is an enforceable standard.

Validator	How it works
<code>conventional_commits.py</code>	Reads <code>.git/COMMIT_EDITMSG</code> . Regex-checks the subject line against <code>type(scope) : description</code> format, validates max 72 chars, imperative mood heuristic, and <code>Agent-Session:</code> trailer format if present. Runs at <code>commit-msg</code> stage.
<code>british_english.py</code>	Receives file paths from pre-commit. Scans lines outside code blocks for American spellings (color, behavior, organize, center, license). Brittle: the word list is hardcoded — new pairs need adding manually.
<code>ears_notation.py</code>	Receives file paths. Matches lines containing “shall” against six EARS patterns (<code>THE x SHALL</code> , <code>WHEN y</code> , <code>THE x SHALL</code> , etc.). Only triggers on files under <code>artefacts/product/</code> .
<code>design_system.py</code>	Scans <code>frontend/src/</code> for CSS violations: multiple CSS files, Tailwind concrete colour classes, direct <code>@heroicons</code> imports outside the barrel, and off-scale spacing tokens. Hardcoded to <code>frontend/src/</code> relative to repo root.
<code>metrics_logging.py</code>	Receives JSONL file paths. Validates each line has required fields (<code>ts</code> , <code>task</code> , <code>agent</code> , <code>event</code> , <code>tokens</code>), valid event types, and correct token source annotations.
<code>supabase_boundary.py</code>	Receives file paths. Regex-checks for <code>import supabase</code> or <code>from supabase import</code> outside the database service.
<code>framework_docs_staleness.py</code>	Queries <code>git diff --cached</code> for staged <code>context/</code> files. If any are found, blocks unless <code>agentic-framework-reference.md</code> is also staged; warns if <code>agentic-framework.md</code> is missing. <code>[docs-ok]</code> in the commit message bypasses the check (auditable via <code>git log --grep='docs-ok'</code>). Only hardcoded exclusion: the two framework docs themselves, to avoid circular triggering.

Consistency compounds. Drift taxes. The templates and validators exist to keep that equation favourable over time.

Limitations

This is not an application architecture. It does not prescribe service boundaries, data models, or UI patterns. It prescribes how agents coordinate whilst building those things.

Rule compliance is probabilistic. Injecting a rule into a spawn prompt does not guarantee the agent will follow it. Models sometimes ignore injected instructions, particularly under context pressure. Validators catch some violations post-hoc, but real-time enforcement during agent execution is not currently possible.

Personal agent settings and memories undermine framework authority. Most agent runtimes load user-level configuration before project-level files. Claude Code reads `~/ .claude/CLAUDE.md` and accumulated session memories before it reads the project's `CLAUDE.md` or `AGENTS.md`. Cursor reads user-scoped rules and settings before workspace rules. This means personal preferences, habits, and stale memories can silently override framework rules, standards, and workflows. The framework cannot enforce its operating model if the runtime has already loaded contradictory instructions at higher precedence.

This is not a replacement for human judgement. Human judgement sits at review gates, design approvals, escalation boundaries, and framework evolution decisions. The framework automates the mechanical coordination so that human attention is spent governing above the loop rather than embedded in every step.

This is not the only viable approach. Simpler configurations (a single well-prompted agent with good context) can be effective for smaller or less demanding projects where supervision overhead is acceptable. A swarm model (multiple agents exploring different approaches in parallel, with the best result selected) trades coordination overhead for exploration breadth and may suit problems where the optimal solution is unclear. This framework defaults to a hive model because production-quality work benefits more from coordinated specialisation than parallel exploration.

Further Reading

The framework's design principles are not novel; they adapt ideas from distributed systems, software engineering, and domain-driven design. The following references ground each principle in its source material.

Specialisation and bounded responsibilities: - Newman, S. (2021). Building Microservices: Designing Fine-Grained Systems, 2nd ed. O'Reilly. Chapters on modelling service boundaries around bounded contexts, and the trade-offs between cohesion and coupling in fine-grained systems. - Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley. The original treatment of bounded contexts: explicit boundaries within which a model applies, with well-defined interfaces between them.

Explicit orchestration over implicit choreography: - Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning. Chapters 4-5 on saga orchestration vs choreography: when a central coordinator should manage workflow state and when event-driven coordination is preferable. - Helland, P. (2007). "Life beyond Distributed Transactions: an Apostate's Opinion." CIDR 2007. The case for managing state through explicit workflow and messaging rather than distributed transactions, and why uncertainty at a distance must be designed for rather than abstracted away.

Durable context and externalised state: - Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly. Chapter 11 on stream processing, event sourcing, and deriving state from immutable logs. The principle that state should be reconstructable from a durable record rather than held only in memory.

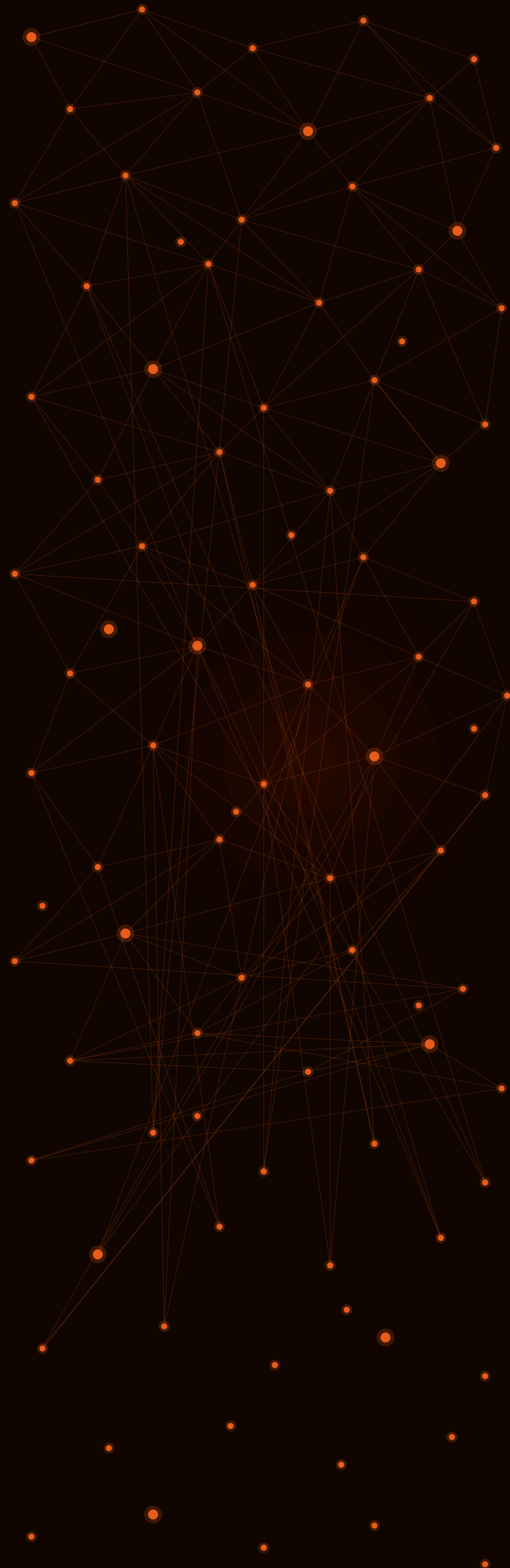
Verification over assertion: - Beck, K. (2002). Test-Driven Development: By Example. Addison-Wesley. The foundational TDD cycle (red, green, refactor) and the discipline of writing executable verification before implementation. - Fowler, M. (2007). "Mocks Aren't Stubs." martinowler.com. The distinction between state verification (checking outcomes) and interaction verification (checking method calls), and why observable evidence is more trustworthy than reported behaviour.

Portability and tool-agnostic design: - Fowler, M. and Lewis, J. (2014). "Microservices." martinowler.com. The principle of smart endpoints and dumb pipes: keeping logic in the services (agents) rather than coupling it to the transport or runtime infrastructure.

Strategic principles and operating model: - Sinharay, A. (2026). New DevX Vision. [context/docs/vision.md](#). Establishes the three strategic principles (Elevate the Next Constraint, Everything as Code, Humans Above The Loop) and three shifts in perspective (Sequence Speed and Rigour, Work the Way Agents Work, Navigate Don't Arrive) that this framework implements. Covers the four-phase workflow, context hierarchy, risk-based oversight, and the autonomy horizon. - Sinharay, A. (2026). Journey to New DevX. [context/docs/journey.md](#). The implementation companion to the Vision. Introduces the operating triangle (execution time, token cost, autonomy horizon), AgentOps as an operational discipline, the comprehension gap, the three maturity stages (DevX as Supervisor, Reviewer, Orchestrator), and context infrastructure. This framework implements those concepts as a portable directory structure. - Sinharay, A. (2026). Roadmap: Planning the Journey to New DevX. [context/docs/roadmap.md](#). Adoption planning and measurement companion. Covers readiness assessment, present-forward vs future-back postures, stage benchmarks, the RACER operating loop for constraint elevation, and the metrics taxonomy (Focus, Speed, Predictability, Quality, Durability).

Part 2

Framework reference



Framework reference — Contents

Agentic Framework Reference	20
Framework Topology	21
Source of Truth Hierarchy	22
Root-Level Framework Files	24
Standards	25
Rules	27
Agents	29
Workflows	34
Orchestration Patterns	46
Templates	48
Scripts and Automation	50
Portability and Framework Adapters	53
How the Parts Connect	56
Reading Paths	57

Agentic Framework Reference

The companion technical paper explains why the framework exists and the design principles behind it. This reference explains what the framework contains and how each part works. It is structured so that either document can be read first, or independently.

Use this document when you need to answer questions such as:

- What is this directory for?
- How does rule injection work at spawn time?
- What does the default workflow look like phase by phase?
- How would I port an agent definition to a different orchestration system?
- How do I measure whether the framework is working?

Each section covers one structural area of the framework: what it contains, how it operates, and where to make changes.

The companion paper develops the why and the trade-offs behind those choices. As you read the catalogue below, the same posture shows up repeatedly: explicitness (written phases, gates, paths, recovery—not tacit convention); durability (artefacts and handoffs—not chat memory); reuse (templates, shared agents, generated registries—not one-off prose); portability (Markdown and YAML at the core—not a single vendor runtime); and verification (validators, tests, reviews, gates, telemetry—not unevidenced claims). For the full argument, see `context/docs/agentic-framework.md`.

This reference summarises the tree on disk. If anything here disagrees with an authoritative file (a workflow YAML, an agent definition, a rule, or a generator script), trust the file and treat the mismatch as documentation drift to fix.

Framework Topology

The framework is organised as a layered directory structure inside `context/`. The layers are ordered below by how frequently each is accessed during execution. Layers that touch every agent spawn sit at the top; layers consulted once per task or occasionally sit at the bottom.

Layer	Location	Responsibility	Access Frequency
Rules	<code>context/rules/</code>	State what must happen: short, binary, injectable constraints	Every agent spawn
Agents	<code>context/agents/</code>	Define who: specialised roles with frontmatter linking rules and standards	Every agent spawn
Templates	<code>context/templates/</code>	Define what shape: standardised formats for reviews, tasks, handoffs, design docs	Every output
Standards	<code>context/standards/</code>	Explain why and how: engineering rationale for coding, testing, security, documentation	On demand
Scripts	<code>context/scripts/</code>	Enforce everything above: validators, generators, git hooks	Every commit
Workflows	<code>context/workflows/</code>	Define when: phase order, dependencies, gates, outputs, recovery	Once per task
Docs	<code>context/docs/</code>	Provide further reading: design rationale, strategic context, orchestration patterns, workflow guides	Human reference
Personas	<code>context/persona/</code>	Provide voice: writing style for human-facing content only	Content workflows only

The layering determines where changes should be made. To change what agents do, edit a workflow. To change how they do it, edit a standard or rule. To change who does it, edit an agent definition. To change what the output looks like, edit a template. Nothing bleeds across boundaries unless explicitly designed to.

The design pattern is consistent throughout:

- ▶ define the role or rule
- ▶ explain how to use it
- ▶ standardise the output
- ▶ validate the result

Source of Truth Hierarchy

Not all framework files carry equal authority. Some define behaviour; others project or summarise it.

Tier	Examples	Role
Authoritative source	context/agents/*.md, context/workflows/*.yaml, context/rules/*.mdc, context/standards/*.md, context/templates/*.md, context/persona/*.md	These define how the framework behaves. Edit these to change it.
Derived projection	AGENTS.md, CLAUDE.md	These surface framework information in runtime-friendly form. Never hand-edit; regenerate from source.
Operator guidance	context/docs/*.md, README.md	These explain how to use, extend, or evaluate the framework.
Enforcement	context/scripts/validators/*, context/scripts/prepare-commit-msg.*	These keep the repository aligned with the design.

The principle: change the authoritative source, let derived projections follow. Contributors should be able to look at any file and know whether it is the right place to make a change or whether the real change needs to happen upstream.

Antipattern: Personal Agent Settings and Memories

The source of truth hierarchy above governs files inside the repository. But agent runtimes introduce a precedence layer the repository cannot control: user-level configuration and accumulated memories that load before project-level files, silently overriding framework authority.

How it works. Claude Code reads `~/.claude/CLAUDE.md` and accumulated session memories before it reads the project's `CLAUDE.md` or `AGENTS.md`. Cursor reads user-scoped rules (in `~/.cursor/rules/`) and user settings before workspace-level rules. Other runtimes follow similar patterns. The result is a shadow configuration layer that sits above the framework's authoritative sources in the effective precedence order:

Precedence	Source	Controlled by
1 (highest)	User memories (auto-accumulated)	Runtime
2	User-level config (<code>~/.claude/CLAUDE.md</code> , <code>~/.cursor/rules/</code>)	Individual contributor
3	Project-level derived files (<code>CLAUDE.md</code> , <code>AGENTS.md</code>)	Framework (generated)
4	Authoritative sources (<code>context/</code>)	Framework (versioned)

Anything at precedence 1 or 2 that contradicts precedence 3 or 4 wins — and the framework has no mechanism to detect or prevent it.

Why this breaks the framework. The framework’s design assumes that `context/` is the authoritative operating model and that derived projections (`AGENTS.md`, `CLAUDE.md`) faithfully represent it at runtime. Personal settings and memories violate that assumption. A contributor whose agent runtime has memorised “always use `pip install`” will have that memory override the framework’s `python-environment.mdc` rule requiring `uv add`. A user-level `CLAUDE.md` that says “keep responses concise” can override the framework’s handoff requirements. These overrides are invisible to other contributors, to review agents, and to the framework’s validators — the pre-commit hooks check committed files, not what was in the agent’s context window.

Symptoms. Inconsistent agent behaviour across contributors working on the same repository. Rules that are correctly defined and injected but not followed. Agents that produce output in a style or format that does not match any framework standard. Validators passing at commit time on rules that were violated during execution (because the violation was in approach, not in the committed artefact). Debugging these failures is difficult because the cause lives outside the repository.

Mitigations.

- ▶ **Disable agent memories** for repositories governed by this framework. In Claude Code: do not use `/memory` and clear any existing memories that overlap with framework concerns. The framework’s durable context (handoffs, task files, standards) replaces what memories are meant to provide, with the advantage of being versioned, shared, and inspectable.
- ▶ **Avoid user-level configuration that overlaps with framework concerns.** If `context/rules/python-environment.mdc` governs Python tooling, do not also set Python tooling preferences in `~/.claude/CLAUDE.md` or `~/.cursor/rules/`. User-level configuration should be limited to genuinely personal, project-independent preferences (editor keybindings, display settings) that do not intersect with anything the framework manages.
- ▶ **Audit when behaviour diverges.** If an agent ignores a correctly injected rule, check user-level configuration and memories before assuming the rule is broken or the model is non-compliant. The most common cause is a higher-precedence instruction contradicting the rule.
- ▶ **Document the requirement.** Onboarding instructions for contributors should explicitly state that personal agent configuration must not overlap with framework-managed concerns, and that agent memories should be disabled or cleared.

The framework cannot enforce this boundary — it sits below the precedence layer it needs to control. This is a governance and onboarding discipline, not an automated check.

Root-Level Framework Files

AGENTS.md

Auto-generated registry of workflows and agents. It acts as the root-level entry point for agent runtimes and humans. The generator script (`context/scripts/generators/generate_agents_md.py`) reads every workflow YAML and every agent definition, then produces `AGENTS.md` as a single derived file containing the phase sequence, agent dispatch rules, spawn patterns, and state recovery procedures. If an agent definition references a rule that does not exist, or a workflow references an agent with no definition, the mismatch surfaces at generation time rather than at runtime.

Editing `AGENTS.md` by hand is always wrong. Edit the sources and regenerate.

CLAUDE.md

Root agent guidance surfaced to the runtime. In this repository it points to `AGENTS.md`, making the generated registry the durable entry point instead of duplicating framework detail in multiple places.

README.md

Repository-level onboarding and setup document. It complements `context/README.md`: the root README explains the repository; `context/README.md` explains the framework. This split keeps newcomer setup instructions close to the repository root whilst leaving framework theory in the portable context package.

Standards

Standards are detailed reference material stored in `context/standards/`. They are intentionally longer and more explanatory than rules. An agent consults a standard when it needs deeper guidance than a rule provides: the reasoning behind a technology choice, the criteria for a good handoff, the philosophy behind the testing approach.

How Standards Are Used

Each agent definition lists the standards it depends on in its frontmatter. When the orchestrator spawns an agent, it includes the standard references in the spawn prompt. The agent reads the relevant standards before starting work. Standards are loaded on demand rather than injected wholesale, because a single standard can run to 2,000-5,000 tokens and loading all of them would overwhelm the context window.

The Rules-Standards Split

Rules and standards serve different purposes, and the separation is the framework's most consequential structural decision.

Rules are under 200 tokens each. They state what must happen: “use `uv run pytest`, not bare `pytest`”; “agents do not commit; the orchestrator commits on their behalf.” They are designed for repeated injection into spawn prompts without significant context window cost. A typical agent spawn injects 5-8 rules at ~100-200 tokens each, plus the agent definition at ~500-1,000 tokens, totalling roughly 1,500-2,500 tokens of framework overhead.

Standards run to hundreds of lines. They explain why the framework uses `uv`, how TDD phases relate to each other, what constitutes a good handoff, and when to escalate versus assume.

An agent executing a straightforward Python task needs the 15-token rule that says “run tests with `uv run pytest`.” A reviewer evaluating whether the tests are sufficient needs the full testing standard. The framework serves both without forcing either to carry the other's weight.

Standards Catalogue

`context/standards/README.md` is the lightweight index for humans navigating the folder. The tables below describe what each standard is for when you are editing behaviour or answering an agent's “where do I read more?” question. They intentionally omit the README itself, which is navigation—not a behavioural standard agents load from frontmatter.

Foundational

Standard	Purpose
<code>agent-standards.md</code>	Defines how agents should behave, recover state, use tools, and coordinate. The behavioural contract around agents, not just their roles.
<code>context-framework.md</code>	The framework's theory of context: reference vs actionable context, priority, handoff format, terse formats, and context quality. A foundational design text.
<code>workflow-standards.md</code>	Shared workflow conventions, invocation practices, and orchestration rules. The operating model that spans multiple workflow YAML files.

Technical

Standard	Purpose
<code>coding-standards.md</code>	Language and implementation guidance for code quality. Patterns, error handling, naming.
<code>tech-standards.md</code>	Primary technology stack reference. Tools, libraries, deployment targets, environment setup.
<code>testing-standards.md</code>	Testing philosophy and practice, including TDD expectations, coverage requirements, and phase-based thresholds.
<code>security-standards.md</code>	Security guidance for design, implementation, and review.
<code>tech-mobile-standards.md</code>	Mobile-specific technical guidance, separated to avoid bloating the main technology standard.

Quality and Documentation

Standard	Purpose
<code>doc-standards.md</code>	Documentation structure, quality, archival, diagram, and maintenance guidance.
<code>build-standards.md</code>	How build-phase work should be structured and assessed.
<code>visual-standards.md</code>	Guidance for design consistency, UI quality, and visual review concerns.
<code>12-factor-principles.md</code>	Maps framework work onto twelve-factor application principles.
<code>LESS-Engineering-Principles.md</code>	Captures the LESS principles used by review agents and design discussions.

Rules

Rules are short, enforceable constraints stored in `context/rules/` as `.mdc` files. Each rule addresses a real failure mode: something that breaks, drifts, or degrades if the rule is not followed. They are designed for injection into agent prompts and for validator-backed enforcement.

How Rules Work

Each rule file has YAML frontmatter with three fields:

```
---
description: Python environment setup and command execution
globs: ["**/*.py", "**/pyproject.toml", "**/requirements.txt"]
alwaysApply: false
---
```

`description`: A short summary of what the rule covers.

`globs`: File patterns that determine when this rule applies. The orchestrator matches the task's files against these globs and injects only the rules whose patterns match.

`alwaysApply`: If `true`, the rule is injected into every agent spawn regardless of file scope.

The body of the rule is concise instruction text, typically structured as a table of correct vs incorrect usage, followed by a brief rationale. The entire rule stays under 200 tokens.

Example: Python Environment Rule

Action	Correct	Wrong
Run tests	<code>uv run pytest</code>	<code>pytest</code>
Install package	<code>uv add package</code>	<code>pip install package</code>
Run script	<code>uv run python script</code>	<code>python script.py</code>

The rationale section explains that bare `python` or `pytest` commands fail because no virtual environment is activated and dependencies are not available. `uv run` automatically uses the project's virtual environment, ensures dependencies are installed, and respects `.python-version`.

Rules Catalogue

Environment and Tooling

Rule	Purpose
<code>python-environment.mdc</code>	Enforces <code>uv</code> for all Python commands
<code>typescript-environment.mdc</code>	Enforces <code>yarn</code> for package management, <code>yarn dlx</code> for one-off tools
<code>bash-environment.mdc</code>	Prevents unsafe or inconsistent shell usage; pushes work toward approved tools
<code>mermaid-environment.mdc</code>	Constrains Mermaid diagram syntax, layout direction, and node label conventions
<code>supabase.mdc</code>	Defines constraints around Supabase-related usage

Quality and Process

Rule	Purpose
tdd-workflow.mdc	Enforces RED then GREEN then BLUE discipline
quality-gates.mdc	Encodes non-negotiable review and release criteria
output-locations.mdc	Keeps artefacts in the correct directories
handoff-hygiene.mdc	Enforces durable, useful handoffs
escalation.mdc	Defines when agents may assume, flag, or escalate
git-commits.mdc	Standardises commit structure and agent session metadata
metrics-logging.mdc	Constrains how workflow and performance signals are recorded
no-ai-slop.mdc	Eliminates AI-generated writing tics from persona-driven documentation

Architecture, Design, and Safety

Rule	Purpose
architecture-fidelity.mdc	Prevents implementation drift from approved architecture
visual-fidelity.mdc	Prevents drift from approved UI and visual design
ui-component-reuse.mdc	Encourages reuse and discourages needless UI duplication
type-safety.mdc	Protects typed contracts and discourages unsafe shortcuts
browser-automation.mdc	Constrains browser-based test and automation behaviour
secrets-management.mdc	Prevents unsafe handling of secrets and credentials

Language and Requirements Hygiene

Rule	Purpose
british-english.mdc	Enforces consistent spelling in framework outputs
EARS-notation-requirements.mdc	Enforces the chosen requirements notation

Writing New Rules

A candidate for a new rule should pass three tests:

1. Is it binary? Can you definitively say “followed” or “not followed”?
2. Does it break things? Not just a style preference; actual failures result from ignoring it.
3. Is it actionable? Can you give clear DO/DON'T commands?

If all three are true, create a rule in `rules/*.mdc` and keep it under 200 tokens. If the topic requires nuance, explanation, or judgement, it belongs in a standard, not a rule.

Agents

Agent definitions are markdown files with YAML frontmatter and an instruction body, stored in `context/agents/`. They are the role catalogue of the framework: each file defines a single specialist with a bounded responsibility, a curated set of rules and standards, and clear instructions for how to execute its work.

The repository ships one orchestrator plus eighteen delegatable specialists (nineteen role files in total, excluding `README.md`). The roster below groups them by concern; `context/agents/README.md` lists the same roles with recommended model tiers for operators who tune cost versus capability. Which agent runs in which phase is never implied by the roster alone; it is declared in `context/workflows/*.yaml` and reflected in regenerated `AGENTS.md`.

Agent Definition Format

Every agent follows this structure:

```
---
name: python-coder
description: Writes production Python code with testing in mind.
model: sonnet
standards:
  - tech-standards.md
  - coding-standards.md
  - security-standards.md
  - doc-standards.md
rules:
  - python-environment.mdc
  - tdd-workflow.mdc
  - type-safety.mdc
  - git-commits.mdc
  # ... additional rules
# Optional - see "Tool and MCP surfaces" below
# mcp_tools:
#   - context7
---

You are an expert Python engineer. Your job is to write clean,
testable, production-grade Python code.

## Required Standards (Read First!)
1. context/standards/tech-standards.md
2. context/standards/coding-standards.md
...

## Workflow
1. Read standards
2. Read task context
3. Implement
4. Run tests
5. Write handoff
```

name: Identifier used in workflow YAML and spawn prompts.

description: Brief role summary. Helps the orchestrator select the right agent.

model: Recommended model tier (e.g. opus, sonnet, haiku). The orchestrator uses this as guidance, not a hard constraint.

standards: List of standard files the agent should read before starting work. These are loaded on demand, not injected automatically.

rules: List of rule files that apply to this agent's work. The orchestrator injects these into the spawn prompt.

`mcp_tools` (optional, common on specialists here): MCP server identifiers the author expects the runtime to attach for that role (for example library docs or database inspection). Other products use `allowed_tools` or an equivalent allow-list instead. Neither field executes by itself—it documents intent for whoever configures the runtime.

The body below the frontmatter is the agent’s system prompt: role definition, workflow steps, output expectations, and any role-specific constraints.

Tool and MCP surfaces (runtime vs frontmatter)

Agent YAML declares what a role should be able to reach; the orchestrator runtime (Cursor, Claude Code, a headless worker, and so on) still decides which tools and MCP servers actually exist, which paths are writable, and which invocations require human approval. Those layers live outside `context/agents/`—often in workspace or user settings that are not committed. In Cursor, permissions and MCP wiring frequently sit in `.cursor/settings.local.json` (typically git-ignored), so two machines with the same checkout can behave differently.

When frontmatter and local configuration diverge, the failure mode is friction: endless approval prompts, subagents that cannot attach a listed MCP server, or specialists that appear “fully defined” in markdown but never receive the tools their prompt assumes. Treat that as a deployment alignment problem: update the local settings (or your organisation’s standard Cursor profile) when you add `mcp_tools`: entries, add a new agent, or enable a new server—and keep frontmatter honest about what is truly required versus optional.

Rule injection and file scope (see `context/agents/orchestrator.md`) remain mandatory regardless of tool lists; missing rules cause as much pain as missing MCP entries.

How the Orchestrator Resolves an Agent

When spawning an agent, the orchestrator:

1. Reads the agent definition from `context/agents/{name}.md`
2. Extracts the `rules` list from frontmatter
3. Glob-matches the task’s files against each rule’s scope to confirm applicability
4. Injects the applicable rules into the spawn prompt alongside the agent’s system prompt
5. Constrains the agent to a defined file scope so it cannot make changes outside its boundaries
6. Constructs the full prompt using the task prompt template (`context/templates/task-prompt-template.md`)

Agents do not commit to git. Only the orchestrator commits, one at a time, preventing ref-lock collisions when parallel agents finish simultaneously.

The Agent Roster

The framework defines 18 specialised agents, organised by kind of judgement.

Core Coordinator

Agent	Purpose
<code>orchestrator</code>	Coordinates the system, resolves rules, assigns file scope, validates phase outputs, manages quality gates

Product and Discovery

Agent	Purpose
product-expert	Elicits problem understanding and discovery insight from the user
product-owner	Turns clarified needs into structured requirements and stories

Design

Agent	Purpose
solution-architect	Designs architecture, boundaries, and data flow
database-designer	Designs schemas, relationships, and migrations
api-designer	Designs APIs and contract artefacts
ui-designer	Designs UI structure, component behaviour, and user flow
visual-designer	Produces visuals, mockups, and visual review input

Implementation and Test

Agent	Purpose
python-coder	Implements Python production work
typescript-coder	Implements TypeScript and frontend work
functional-tester	Writes and runs tests in the TDD flow
ui-tester	Executes browser-based UI verification

Review and Governance

Agent	Purpose
tech-lead	Acts as the principal architecture and quality gate reviewer
code-reviewer	Performs implementation-focused review for bugs, risks, and maintainability
principles-reviewer	Reviews designs and code against higher-level engineering principles
security-tester	Reviews for vulnerabilities and security risks

Operations and Maintenance

Agent	Purpose
devops	Handles deployment and infrastructure-related work
documentation	Cleans up, archives, and maintains human-facing documentation

Agent	Purpose
workflow-analyst	Reviews process effectiveness and supports retrospectives

Creating a New Agent

1. Copy `context/templates/agent-template.md` to `context/agents/{name}.md` (some deployments also keep a `context/agents/TEMPLATE.md` copy; the templates directory is the canonical scaffold in this tree)
2. Set name, description, model in frontmatter
3. List applicable rules and standards in frontmatter
4. Write a concise body with role definition, workflow steps, and output expectations
5. Add the agent to the appropriate workflow YAML phase
6. Regenerate `AGENTS.md`

Writing Personas

Personas are stored in `context/persona/` and provide voice guidance for human-facing content. They are used by the `documentation` agent when writing blogs, technical papers, user guides, and marketing materials. They are explicitly not used for technical handoffs, API documentation, architecture documents, or internal artefacts.

When to Use a Persona

Use a persona for: blog posts, articles, thought leadership, technical papers, user-facing guides for non-technical audiences, marketing materials, conference talks.

Do not use a persona for: technical handoffs (README, HANDOFF.md, build notes), API documentation, code references, architecture documents, design specifications, internal artefacts (requirements, tasks, review reports), code comments, error messages.

Available Personas

Technical Writer

File: `context/persona/technical-writer.md` Character: Amara Osei. Eight years at Stripe documenting payment APIs, four years at Hashicorp writing infrastructure guides. Use for: Technical guides, framework documentation, practitioner reference material.

Key traits: leads with the problem, not a hook. Explains the mechanism step by step. Working examples with inputs and outputs. Names trade-offs and costs directly. Structured for reference use with scannable headings and front-loaded information. No filler.

Opinionated Blogger

File: `context/persona/opinionated-blogger.md` Character: Dr. Sarah Chen. PhD from MIT, 10 years as principal engineer at Google. Use for: Blog posts, opinion pieces, thought leadership.

Key traits: provocative opening hooks. Conversational with contractions and parenthetical asides. Personal anecdotes as entry points. Self-deprecating honesty. Question-driven framing. Memorable one-liners. Short paragraphs, 2-4 minute read times.

Editor

File: `context/persona/editor.md` Use for: Refining and polishing existing content. Checking voice consistency, removing AI slop, verifying British English.

Expert Reviewer

File: `context/persona/expert-reviewer.md` Use for: Reviewing technical accuracy and completeness of human-facing content.

Selecting a Persona

The selection depends on audience and content type:

- ▶ Writing a practitioner guide with code examples? Technical writer.
- ▶ Writing a blog post or opinion piece? Opinionated blogger.
- ▶ Writing for a non-technical audience about high-level concepts? Opinionated blogger.
- ▶ Writing implementation details for developers? Technical writer.
- ▶ Polishing an existing draft? Editor.
- ▶ Checking technical claims? Expert reviewer.

All personas use British English throughout (colour, optimise, behaviour, whilst, programme, organisation).

Workflows

Workflows are YAML files in `context/workflows/` that encode phase dependencies, agent assignments, quality gates, validation criteria, state recovery procedures, and execution rules. Storing orchestration logic in versionable YAML rather than ad hoc conversation makes it inspectable, diffable, reviewable, and recoverable after context compaction.

Workflow	File	Purpose
design	design.yaml	Discovery through design review; run before build
build	build.yaml	Primary TDD loop: plan, red, green, blue, review, regress, docs cleanup
prototype	prototype.yaml	Fast iteration, no quality gates; rewrite before production
deploy	deploy.yaml	GCP cloud deployment after local verification
bugfix	bugfix.yaml	Empirical reproduction, diagnosis, fix, verification
full-test	full-test.yaml	Complete regression across all modules
content	content.yaml	Human-facing writing with personas
continuous-improvement	continuous-improvement.yaml	Incident response and retrospective analysis (framework evolution)
retrospective	retrospective.yaml	Standalone process review and pattern identification

Design (design.yaml)

Purpose: Discovery through design review; run before `build.yaml`.

Pattern: Six phases, one approval gate at the end (design-review).

Phase	Agents (pattern)	Role
discovery	product-expert then product-owner (sequential)	Discovery notes → formal requirements and user stories in <code>artefacts/product/</code>
design-architecture	solution-architect	Architecture, logical API contracts, data model, component design docs
design-contracts	database-designer and api-designer (parallel)	Physical schema and migrations; OpenAPI with HTTP semantics
design-ui	ui-designer	Component specs, tokens, wireframes, user flows
design-visuals	visual-designer	Visual assets under <code>artefacts/design/visuals/</code>

Phase	Agents (pattern)	Role
design-review	tech-lead → code-reviewer → principles-reviewer → visual-designer → security-tester (sequential gate)	Design approval; tech-lead must approve before the next reviewer runs

Design gate (design-review): zero architecture blockers; zero design security issues; complex components need a design doc per `design_doc_standard` in the workflow file.

Build (`build.yaml`)

Purpose: Primary TDD implementation pipeline after design approval.

Pattern: Subtask-driven TDD with automated coverage gate, per-sprint review, local deployment, scoped regression, documentation cleanup, and two final approval gates. Runs against `artefacts/build/subtask-plan.yaml` (subtasks, file scopes, acceptance criteria). Canonical phase order and validation live in the YAML; this is a digest.

Sixteen phases (plus a sprint loop that repeats for each sprint in `subtask-plan.yaml`). High-level shape:

```
task-planning → tasks-review (gate) → schema-migration →
  [ per sprint: tdd-red → test-plan-review → tdd-green → coverage-gate (gate) →
    tdd-blue → sprint-review ] →
local-deployment → unit-regression → integration-regression → e2e-regression →
docs-cleanup → quality-review (gate) → final-holistic-review (gate)
```

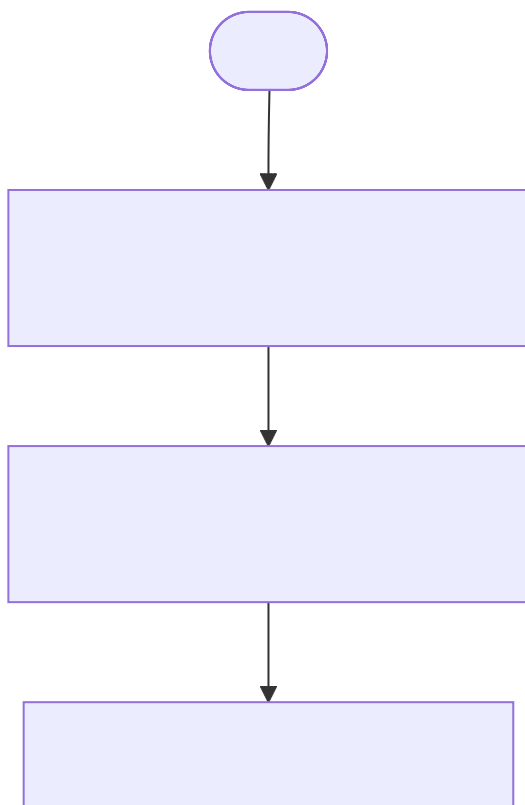
Phase	Role
task-planning	tech-lead decomposes work into subtasks with verifiable acceptance criteria and file scopes (<code>artefacts/build/subtask-plan.yaml</code>).
tasks-review	tech-lead, solution-architect, principles-reviewer in parallel; gate on subtask plan before implementation.
schema-migration	devops writes and applies migrations when the sprint changes schema (skippable if no schema work).
tdd-red	functional-tester writes failing tests for all ACs.
test-plan-review	code-reviewer checks every AC maps to a failing test before GREEN.
tdd-green	python-coder and typescript-coder in parallel; minimal code to pass tests; no test edits.
coverage-gate	functional-tester; automated gate — at least 95% coverage on changed or new files only; fail sends work back to RED.
tdd-blue	Both coders in parallel; refactor only; tests stay green.
sprint-review	code-reviewer, ui-tester, security-tester in parallel; browser screenshots for new UI; findings feed the next sprint.
local-deployment	devops local apply, smoke test, rollback notes.

Phase	Role
unit-regression	functional-tester on changed modules (95% on changed scope).
integration-regression	functional-tester on changed interfaces.
e2e-regression	ui-tester and code-reviewer in parallel; screenshots and design compliance for changed workflows.
docs-cleanup	documentation archives and updates README, API docs, HANDOFF.md, tasks.
quality-review	tech-lead only; code and documentation gate before holistic review.
final-holistic-review	solution-architect, tech-lead, visual-designer, principles-reviewer in parallel; all must approve.

Build gates (see `quality_gates` in `build.yaml`): `tasks-review` (approval), `coverage-gate` (automated threshold), `quality-review` (approval), `final-holistic-review` (approval). Thresholds and metrics are defined in YAML; aggregate coverage for `quality-review` is 95% on new or changed files, not a separate 97% deployment-review phase in this workflow file.

Subtask execution: RED, GREEN, and BLUE run as orchestrated subtasks per stream (Python and TypeScript can progress in parallel with disjoint `file_scope`). Recovery uses `HANDOFF.md`, `artefacts/build/dispatch.md`, and `subtask-plan.yaml` status fields (see `state_recovery` in `build.yaml`).

Default delivery (high level)



When to use design and build

Use design then build for production features and anything that will be maintained long-term. Duration scales with sprint count and scope.

Prototype (`prototype.yaml`)

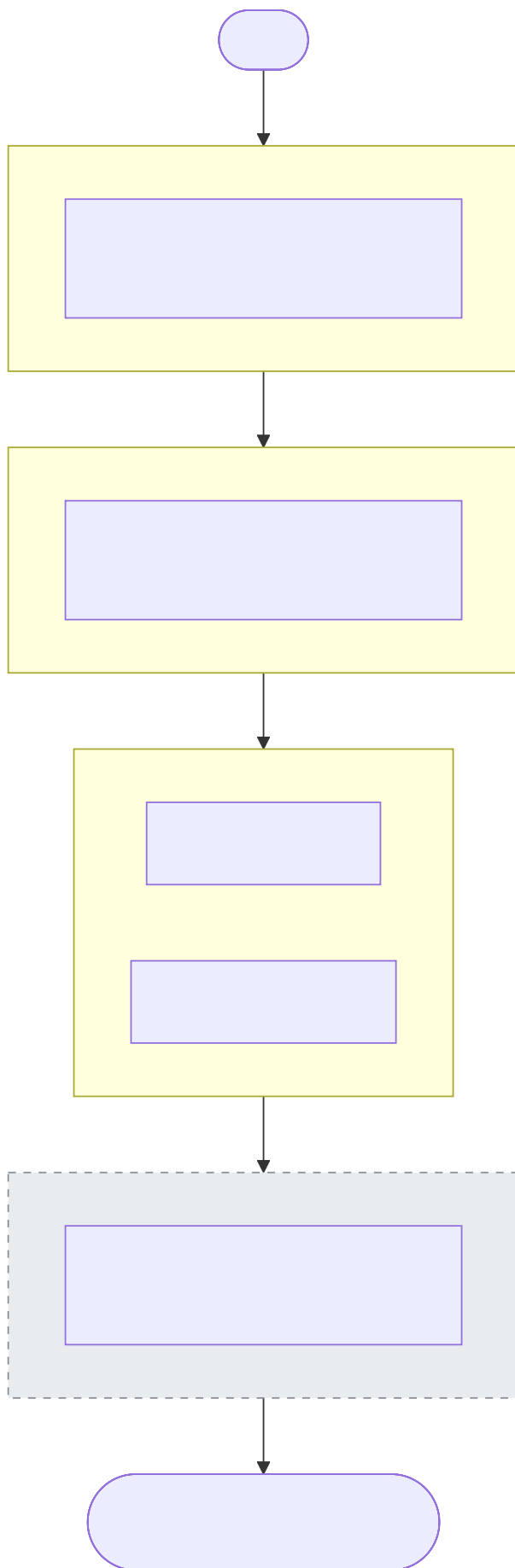
Purpose: Fast iteration without quality gates.

Use for: POCs, experiments, spikes, throwaway code.

The prototype workflow trades rigour for speed: 4 phases, no quality gates, 5 agents.

```
quick-plan → sketch-design → build → validate (optional)
```

Prototype Workflow



Prototype rules

- ▶ Speed over quality. Technical debt is acceptable.
- ▶ No formal reviews, no coverage requirements, no security audits.
- ▶ Tests are optional; smoke tests covering the happy path are sufficient.
- ▶ Hard-coded values, skipped error handling, and incomplete documentation are all acceptable.
- ▶ Mark code clearly as prototype.

Migrating prototype to production

If a prototype validates its hypothesis and should become production code:

1. Archive the prototype on a separate branch
2. Extract learnings: document what worked and what did not
3. Start fresh with design then build (this section's default path)
4. Do not copy-paste prototype code; rewrite with TDD from the beginning

Cleaning up a prototype takes longer than rewriting it properly. The technical debt in prototype code is architectural, not superficial.

Deploy (`deploy.yaml`)

Purpose: GCP cloud deployment and deployment review after `build.yaml` has completed local deployment and verification.

Phase	Agents	Role
<code>gcp-deployment</code>	<code>devops</code>	Terraform, Cloud Run or Firebase Hosting, staging validation, rollback plan under <code>artefacts/gcp/</code>
<code>deployment-review</code>	<code>tech-lead (gate)</code>	Approve production readiness; monitoring and rollback confirmed

`quality_gates` in the YAML require tech-lead approval on `deployment-review`. Criteria include deployment success and coverage sign-off as defined in the file (see `deploy.yaml` for thresholds and notes).

Bugfix (`bugfix.yaml`)

Purpose: Reproduce, fix, and verify defects with empirical evidence (browser screenshots and file-cited causal chains), not feature-sized planning.

Shape: `reproduce (gated; ui-tester and functional-tester in parallel) → fix (failing test first, then python-coder / typescript-coder as needed) → verify (gated; parallel UI and test confirmation)`. Evidence lands under `artefacts/bugfix/`.

Quality gates: Evidence gates on `reproduce` and `verify`—tests passing alone is not sufficient; see YAML for gate metrics and workflow rules (TDD on the fix, commit body records causal chain).

Full-test (`full-test.yaml`)

Purpose: Full-suite testing across all modules—release confidence, not the per-feature changed-scope regressions in `build.yaml`.

Phases (sequential): `full-unit-test → full-integration-test → full-e2e-test → quality-check (tech-lead gate)`. Reports under `artefacts/test-results/`. No code changes in this workflow—failures are reported for follow-up elsewhere.

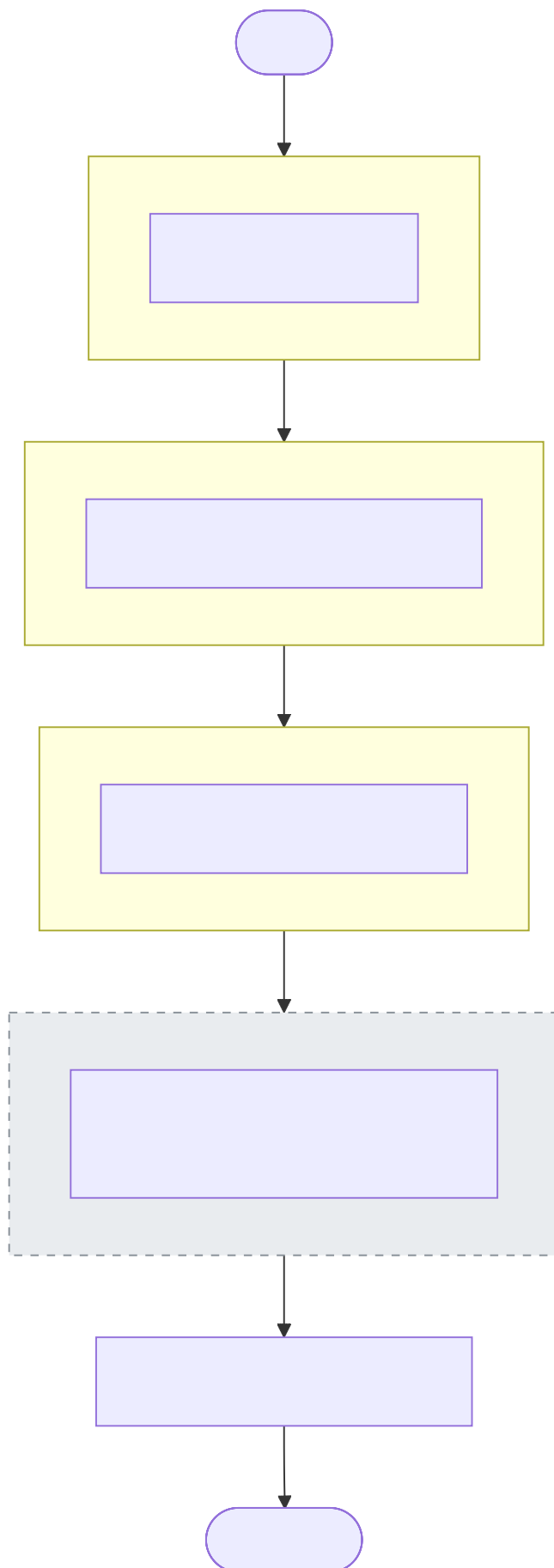
When to run: Before merging a long-running branch to main, after large refactors, or on demand for health checks (see YAML `workflow_rules`).

Content (`content.yaml`)

Five phases, no quality gates (YAML `quality_gates: []`); typical wall time about 2–4 hours. Shape, validation strings, and `workflow_rules` live in `content.yaml`; this subsection is a digest only.

```
research → draft → review → technical-review (optional) → finalize
```

Human-Facing Content Workflow



Phases

Phase	Agent	Persona / role	Primary output
research	product-expert	Audience, purpose, key messages, sources	Research notes (see YAML outputs)
draft	documentation	technical-writer or opinionated-blogger	artefacts/ content/drafts/ {title}-draft.md
review	documentation	editor	artefacts/ content/drafts/ {title}- reviewed.md
technical-review	documentation	expert-reviewer (optional — skip for non-technical pieces)	artefacts/ content/drafts/ {title}-tech- reviewed.md
finalize	documentation	Publication formatting per task	artefacts/ content/ published/ {title}.md

Draft and review phases list concrete checks in YAML (persona voice, British English, opening hook, concrete examples; editorial pass strips patterns covered in `context/rules/no-ai-slop.mdc`). Spawn each phase with `context/templates/task-prompt-template.md`, the persona path, and file scope — long copy-paste invoke blocks belong in the task prompt, not in this reference.

Continuous improvement (`continuous-improvement.yaml`)

Framework improvement and measurement are governed by `context/workflows/continuous-improvement.yaml`, not by ad hoc scorecards. That workflow defines two modes (incident and retrospective), explicit phases, what gets written to disk, and a quality gate in retrospective mode. The `workflow-analyst` agent appears only in retrospective review; other phases are orchestrator- and human-led as the YAML states.

Workflow file and modes

Mode	Trigger	Purpose
incident	Human reports a specific failure	Capture, diagnose, fix, record one incident
retrospective	After deployment, after a sprint, or on demand	Mine logs and git for patterns, discuss with human, fix, record

Workflow rules (from YAML): incident mode is human-triggered; retrospective mode is periodic or on demand; every finding or incident must identify a fixable gap in the framework (rules, standards, templates, scripts), not vanity metrics; in retrospective mode the human decides which findings to implement; incident mode proceeds once diagnosis is sound.

Incident mode (reactive)

Sequential phases:

Phase	Agents	Role
report	None (orchestrator + human)	Append a new entry to artefacts/build/agent-incidents.md: severity, symptom, root cause and fix initially pending
diagnose	None	Trace symptom → mechanism → file → gap with path-level evidence; read configs and code, do not infer from names alone
fix	python-coder, typescript-coder (parallel)	Apply changes to rules, standards, templates, or small orchestration edits; commit after each logical fix
record	None	Update the incident entry with root cause, fix summary, and commit hash; optionally add a portability task to artefacts/build/tasks-context-framework.md

Validation highlights: diagnosis must cite specific files and complete the causal chain; `fix` must satisfy acceptance checks (for example `grep` confirms an anti-pattern removed).

The YAML lists `fix` with `depends_on: [diagnose, discuss]`. Retrospective runs `discuss` (human gate) before `fix`. Incident runs `diagnose` after `report`; the orchestrator should advance to `fix` once diagnosis is complete (the `discuss` phase exists only in retrospective mode).

Retrospective mode (proactive)

Sequential phases:

Phase	Agents	Role
review	workflow-analyst	Read artefacts/build/agent-interruptions.md (questions, tool approvals, escalations), artefacts/build/agent-incidents.md, and git log bodies for Agent-Session telemetry (tokens, duration, agents, dispatch, interactions, approvals injected by <code>prepare-commit-msg</code> and orchestrator). Aggregate to find patterns with counts and examples; each finding must point at concrete files that would change
discuss	None (gate: human approval)	Present pattern → root cause → proposed fix; human chooses what to act on
fix	python-coder, typescript-coder (parallel)	Same as incident mode: implement agreed fixes

Phase	Agents	Role
record	None	Update incidents if relevant; append portability tasks to <code>artefacts/build/tasks-context-framework.md</code> when fixes should propagate to other repos

Quality gate: `discuss` (retrospective only) requires human approval before fixes proceed.

Validation highlights: `git log` parsed for session metrics; interruption and incident logs read; patterns evidenced; no aggregate “health scores” (explicitly disallowed by workflow rules).

Data the retrospective review phase is built to consume

The YAML lists these sources explicitly:

- ▶ `artefacts/build/agent-interruptions.md`: blocking moments (questions, approvals, escalations)
- ▶ `artefacts/build/agent-incidents.md`: prior framework failures
- ▶ `git log` with `%H`, subject, body: reconstruct `tokens=`, `duration=`, `agents=`, `dispatch=`, `interactions=`, `approvals=` from commit trailers

From those, the analyst is expected to answer questions such as: total tokens per agent, sprint, or task; duration per task; which agents cost the most tokens relative to output; token trends; autonomy rate (commits with orchestrator dispatch and zero interactions); distribution of human interactions and approvals. Repeated rule violations, agents over token budget, and incidents with shared root causes are examples of valid pattern classes.

Relationship to other artefacts

Tasks and handoffs (`artefacts/build/tasks.md`, `HANDOFF.md`, `artefacts/shared/handoffs/`) remain useful secondary context when the orchestrator or `@workflow-analyst` widens an investigation, but they are not the primary contract of `continuous-improvement.yaml`; that contract is interruptions log, incidents log, and git telemetry as above.

The agent definition `context/agents/workflow-analyst.md` still describes broader analysis habits (for example task and handoff mining). When in doubt, the workflow YAML wins for phase order, outputs, and validation.

Common fix targets (illustrative)

These are examples of fixable gaps, not a separate scoring methodology:

- ▶ Coverage discipline: tests landed after GREEN instead of strengthening RED or failing `coverage-gate` (see `build.yaml`).
- ▶ Tooling drift: repeated `bash-for-files` or wrong package managers; tighten prompts or rules.
- ▶ Integration cost: missing fixtures or slow local setup; add shared fixtures or documented setup in standards.
- ▶ Parallel misuse: duplicated context without time savings; narrow parallel phases to truly independent work.

Retrospective (`retrospective.yaml`)

Purpose: Standalone proactive review—`workflow-analyst` reviews accumulated operational data, the orchestrator discusses findings with a human (gate), then coders fix and record portability tasks. Not the same file as `continuous-improvement.yaml` (that file adds incident mode and git Agent-Session mining in its retrospective path).

Phase	Agents	Role
review	<code>workflow-analyst</code>	Read agent-interruptions.md, agent-metrics.log, agent-incidents.md; report patterns with file:line evidence
discuss	None (human gate)	Human selects which findings to implement
fix	<code>python-coder</code> , <code>typescript-coder</code> (parallel)	Approved framework changes
record	None	Append portability tasks to artefacts/build/tasks-context-framework.md

See `retrospective.yaml` for `quality_gates`, `workflow_rules`, and `state_recovery`.

Orchestration Patterns

This section documents the four coordination shapes the shipped workflows actually use: single agent, sequential chain, hive (parallel agents on a shared contract, with orchestrator scope rules), and iterative loop (gates and rework). They are vocabulary for reading YAML—not separate runtime features. On disk, behaviour is whatever each workflow file says: phase order, `parallel: true` (see `parallel_planning` in `build.yaml`), `disjoint file_scope` where required, and gates.

Single Agent

One specialist, one task, no dependencies. Use for isolated tasks with clear requirements: fix a specific bug, add a simple feature, write documentation.

Sequential Chain

Agents in strict order, each depending on the previous output. Use for tasks with clear dependencies and quality gates: most of design and build is sequential between phases.

Hive

Parallel agents working against a shared contract (same tests, same subtask plan, same architecture outputs) with orchestrator-enforced scope and merge discipline. The canonical case in `build.yaml` is TDD GREEN and TDD BLUE: `python-coder` and `typescript-coder` run together with `disjoint file_scope` but the same failing-then-passing test suite (`parallel_planning` and `subtask_execution` in the YAML spell this out). `design-contracts` is hive-like too: both designers extend the same architecture handoff. Tasks review, sprint-review, e2e-regression, and final-holistic-review are parallel reviewers on the same body of work—again shared-artefact coordination.

Iterative Loop

Review-fix cycles until approval. Use at quality gates where output may be rejected: design review may require architectural changes before approval.

Pattern Comparison

Pattern	Speed	Token Cost	Coordination	Example in shipped YAML
Single	Fast	Low	None	<code>tdd-red</code> , <code>coverage-gate</code> , many single-agent phases
Sequential	Slow	Low	Linear	Discovery, architecture → contracts → UI chain
Hive	Fast	High	Shared contract + scopes	<code>tdd-green / tdd-blue</code> ; <code>design-contracts</code> ; parallel review phases on one plan or codebase

Pattern	Speed	Token Cost	Coordination	Example in shipped YAML
Iterative	Varies	Medium	Approval loops	design-review, quality gates

How the default path uses patterns

design.yaml

- ▶ Discovery: Sequential (product-expert then product-owner).
- ▶ Architecture: Single agent (solution-architect).
- ▶ Contracts: Parallel hive-like pair (database-designer and api-designer on the same architecture outputs).
- ▶ UI then visuals: Sequential (ui-designer then visual-designer).
- ▶ Design review: Sequential reviewers with iterative return to earlier phases when required.

build.yaml

- ▶ Planning and test planning: Sequential (tech-lead then code-reviewer on the test plan).
- ▶ Tasks review, sprint-review, e2e-regression, final-holistic-review: parallel: true, but agents share the same plan, sprint surface, workflows under test, or release candidate—hive-style coordination.
- ▶ TDD GREEN / BLUE: Hive (parallel coders, shared tests, disjoint file scopes).
- ▶ Coverage gate: Single agent; may loop back to RED.
- ▶ Regression after local deployment: Sequential unit → integration → e2e (the e2e phase may list two agents in parallel with shared workflow scope).
- ▶ Quality review then final holistic review: Gate then parallel approvers on the same implementation.

Choosing a pattern

1. Single isolated task? Single agent.
2. Same plan, tests, or design handoff for everyone? Hive (all parallel: true phases in default path YAML behave this way).
3. Strict A-then-B output dependency? Sequential chain.
4. Output needs iterative refinement? Iterative loop.

Token Optimisation

Parallel execution increases token usage because context is duplicated across agents. Three agents running in parallel typically cost ~20% more tokens than the same three agents running sequentially.

Prefer parallel execution when time is critical, tasks are large (overhead is a small percentage of the total), or agents have different contexts (minimal duplication). Prefer sequential execution when the token budget is tight, tasks are small (overhead dominates), or context is large and identical across agents.

Templates

Templates are stored in `context/templates/` and define standard output shapes for recurring artefacts. The orchestrator selects the appropriate template when constructing a task prompt; the producing agent uses it to structure its output.

`context/templates/README.md` indexes every template file, when to use it, and how it relates to workflows—use it when you are adding a new artefact shape rather than guessing from filenames alone.

Why Templates Exist

Standardised shapes reduce ambiguity for both writers and readers. When every review follows the same structure, or every handoff includes the same fields, downstream agents and humans can consume the output predictably. Automation becomes reliable when the shape is known in advance.

The `commit-message-template.md` is not only for humans formatting commits: it defines the Agent-Session: trailer that `prepare-commit-msg` completes and that `continuous-improvement.yaml` expects the orchestrator to parse from `git log` during retrospective review. The template specifies a machine-readable line with `tool=`, `model=`, `agents=` (role names), optional `duration=`, `dispatch=` (orchestrator or human), `interactions=`, and `approvals=`; the hook appends `tokens=` (input/output token counts). Together those fields support questions in the continuous-improvement workflow (for example cost per agent, autonomy when `dispatch=orchestrator` with zero interactions, and human-approval load). See `context/templates/commit-message-template.md` for the full field table and counting rules.

Available Templates

Template	Purpose
<code>agent-template.md</code>	Starting point for creating a new specialised agent
<code>architecture-template.md</code>	Standard structure for system architecture documents
<code>bugs-template.md</code>	Standard bug tracker format
<code>commit-message-template.md</code>	Standard commit message structure with agent session metadata
<code>design-template.md</code>	Standard design document for complex features or screens
<code>domain-rules-template.yaml</code>	Template for domain-specific constraints
<code>handoff-template.md</code>	Standard agent handoff format
<code>pr-description-template.md</code>	Standard pull request description format
<code>requirements-template.md</code>	Standard requirements structure
<code>review-template.md</code>	Standard review finding format
<code>task-prompt-template.md</code>	Orchestrator prompt scaffold for spawning agents correctly
<code>tasks-template.md</code>	Standard task tracking format
<code>test-dashboard-template.md</code>	Standard structure for test dashboards
<code>user-stories-template.md</code>	Standard user story format

The task prompt template deserves particular attention. It carries rule resolution and file-scope discipline into subagent execution. When the orchestrator spawns an agent, the task prompt template ensures the agent receives its definition file path, resolved rules, immediate context, file scope constraints, and escalation instructions in a consistent structure.

Scripts and Automation

Scripts are stored in `context/scripts/` and form the automation layer that turns the framework from a collection of ideas into an operating system with enforcement.

Generators

`generators/generate_agents_md.py`: Reads every workflow YAML and every agent definition, then produces the derived `AGENTS.md` registry. This is the framework's primary generation pipeline. Run it after changing any workflow or agent definition:

```
uv run python context/scripts/generators/generate_agents_md.py
```

The generator is also an enforcement point. If an agent definition references a non-existent rule, or a workflow references an agent with no definition, the mismatch surfaces at generation time.

Validators

Validators are pre-commit hooks stored in `context/scripts/validators/`. Each validator enforces one or more rules at commit time. A rule without a validator is a suggestion. A rule with a validator is enforceable. Not every rule has automated enforcement yet; validator coverage is expanding.

Validator	What It Checks
<code>conventional_commits.py</code>	Commit message format
<code>british_english.py</code>	British English spelling
<code>ears_notation.py</code>	EARS requirements notation
<code>design_system.py</code>	Frontend design system compliance
<code>metrics_logging.py</code>	Agent metrics JSONL format
<code>supabase_boundary.py</code>	Supabase import boundary
<code>framework_docs_staleness.py</code>	Framework docs updated with context/ changes
<code>validate_agent_definitions.py</code>	Agent definition structural integrity

`conventional_commits.py`

Reads `.git/COMMIT_EDITMSG` (passed as `$1` at the `commit-msg` stage). Regex-checks the subject line against `type(scope): description` — valid types are `feat`, `fix`, `test`, `refactor`, `docs`, `chore`, `perf`; scope must be lowercase alphanumeric with hyphens; max 72 characters. Imperative mood is checked heuristically (flags first words ending in `-ed` or `-ing`). If an `Agent-Session:` trailer is present, validates the `tool=, model=, agents=` triplet format and requires a `Co-Authored-By:` line. Merge and revert commits are skipped.

`british_english.py`

Receives file paths from pre-commit. Scans each line outside fenced code blocks for American spellings: `color`, `behavior`, `organize/realize/recognize/specialize/generalize`, `center`, and `license` (noun form). Lines containing inline code (backticks) or markdown tables are skipped. The word list is hardcoded — new American/British pairs must be added manually.

`ears_notation.py`

Receives file paths from pre-commit. Only runs on files matching `artefacts/product/(requirements|user-stories).md`. For any line containing “shall,” checks it against six EARS patterns: ubiquitous (`THE x SHALL`), event (`WHEN y, THE x SHALL`), state (`WHILE y, THE x`

SHALL), optional (IF y, THE x SHALL), forbidden (THE x SHALL NOT), and complex (WHEN y, IF z, THE x SHALL). Lines inside code blocks, headers, table separators, and list markers are skipped.

design_system.py

Scans `frontend/src/` (hardcoded path relative to repo root via `Path(__file__).parents[3]`). Four checks: only `index.css` allowed (no component-scoped CSS); no Tailwind concrete colour classes like `text-green-500` (use DaisyUI semantic tokens); no direct `@heroicons/react` imports outside the barrel file `components/icons/HeroIcons.tsx`; and spacing tokens restricted to `gap-1`, `gap-2`, `gap-4`, `p-2`, `p-4`, `px-4` (with `md:gap-6/md:px-6` exempted in `PriceHeader` only). Test and spec files are excluded from colour and spacing checks. Bypassed with `SKIP=design-system`. Does nothing if `frontend/src/` does not exist.

metrics_logging.py

Receives file paths from pre-commit. Only validates `.jsonl` files whose path contains `metrics`. Parses each line as JSON and checks for required fields (`ts`, `task`, `agent`, `event`, `tokens`), ISO 8601 timestamp format, event type from a fixed set (`start`, `handoff`, `escalate`, `complete`, `blocked`), token object structure (`in`, `out`, `source`), and that `handoff/escalate` events include a `to` field.

supabase_boundary.py

Receives file paths from pre-commit. Regex-checks each file for `import supabase` or `from supabase import` at any indentation level. Any match outside the database service fails the commit. No path-based exclusion in the script itself — the pre-commit config's `files` pattern controls which files are checked.

framework_docs_staleness.py

Does not receive file paths — runs with `pass_filenames: false`. Queries `git diff --cached` for any staged file under `context/`. If context files are staged, checks whether the two framework docs are also staged. If the reference doc (`agentic-framework-reference.md`) is missing, blocks the commit (exit 1). If the brief doc (`agentic-framework.md`) is missing, warns but passes (exit 0). If the commit message in `.git/COMMIT_EDITMSG` contains `[docs-ok]`, passes unconditionally — the marker is an explicit assertion that the committer assessed docs impact. Auditable via `git log --grep='docs-ok'`. The only hardcoded exclusion is the two framework docs themselves, to avoid circular triggering.

validate_agent_definitions.py

Not wired into pre-commit; run manually. Validates every `.md` file in `context/agents/` (excluding `README.md` and `TEMPLATE.md`). Checks: YAML frontmatter is present and parseable; required fields `name` and `model` exist; every entry in `standards:` points to a file that exists in `context/standards/`; every entry in `rules:` points to a file that exists in `context/rules/`; the body contains no hardcoded absolute paths (`/Users/`, `/home/`). Existence checks are relative to the repo working directory at runtime.

prepare-commit-msg hook (agent tokens on commits)

Files: `context/scripts/prepare-commit-msg.sh` (wrapper that runs `uv run python ../prepare-commit-msg.py`) and `context/scripts/prepare-commit-msg.py` (implementation).

Git calls `prepare-commit-msg` with the path to the commit message draft. The Python script only augments commits that already contain an `Agent-Session:` line (see `context/templates/commit-message-template.md`); if that trailer is absent, it assumes a human-only commit and exits without writing.

What it does

1. Finds Claude Code session telemetry — It does not look up a single hard-coded path name. From `git rev-parse --show-toplevel` it builds `~/.claude/projects/<repo-root-with-each-slash-replaced-by-a-hyphen>` (the same string transform Claude Code uses for project folders in this hook's implementation). Under that directory it lists session subfolders and picks the most recently modified one—so the active session is whichever directory Claude last wrote to, not one global filename shared by every repository. Token lines are then read from that folder's `*.jsonl` files as described below.
2. Sums tokens — Reads the session's main `*.jsonl` plus `subagents/*.jsonl`, accumulating `input_tokens`, cache-related input token fields, and `output_tokens` from each line's `message.usage` object.
3. Uses a watermark file — In that same session directory it reads `.tokens-watermark`, a JSON file `{"in": <int>, "out": <int>}` holding cumulative token totals after the last commit this hook successfully annotated. It computes deltas (`current_cumulative - watermark`) and appends `tokens=<delta_in>/<delta_out>` to the `Agent-Session:` line (replacing an existing `tokens=...` token pair if already present). `duration`, `dispatch`, `interactions`, and `approvals` are not produced by this script—they must be supplied by the orchestrator in the message body per the commit template. After updating the message, the script rewrites `.tokens-watermark` with the new cumulative totals so the next commit records only usage since this one.

Edge behaviour

- ▶ Merge and squash commit sources are skipped entirely (no token injection).
- ▶ If both deltas are zero, the hook does not touch the message or watermark.
- ▶ The watermark lives outside the repository (next to Claude's JSONL); deleting or losing it makes the next run attribute a larger delta (everything since the lost baseline)—operators should treat it as part of session-local state, not something to version in git.

Tests

The scripts directory includes its own test suite in `context/scripts/tests/`. Tests cover validators, generators, and agent definition validation using positive and negative fixture files in `tests/fixtures/`.

Portability and Framework Adapters

The framework is designed to survive changes in agent runtime or orchestration platform. Agent definitions are markdown files. Workflows are YAML. The core logic lives in the system prompt body, which is framework-agnostic. The frontmatter carries runtime-specific metadata (tool names, model tiers) that adapts to each platform.

The code samples in this section are illustrative: they show how to load markdown prompts and map tool names, not a supported SDK matrix. Your runtime may use different client libraries, different tool primitives, or different sandbox rules—keep the portable assets (markdown, YAML, rules, standards) and replace the adapter glue per platform.

Agent Definition Portability

Frontmatter may list `mcp_tools`: (this tree) or `allowed_tools`: (other runtimes)—both are platform-specific hints. The markdown body (role, workflow, constraints) is what ports cleanly. Moving to a new platform means mapping those hints onto that platform's tool and MCP configuration (often a settings file or admin console, not something validators in this repo can see); the prompt itself travels unchanged.

Adapter Examples

LangGraph

Load the agent markdown as a node system message, parse frontmatter for tool bindings:

```
from langgraph.graph import StateGraph
from pathlib import Path

def load_agent(name: str) -> str:
    return Path(f"context/agents/{name}.md").read_text()

def python_coder_node(state):
    agent_prompt = load_agent("python-coder")
    task_prompt = f"{agent_prompt}\n\n## Task\n\n{state['task']}"
    response = llm.invoke(task_prompt)
    return {"output": response}

workflow = StateGraph(State)
workflow.add_node("python-coder", python_coder_node)
```

CrewAI

Load agent markdown as backstory, map tools to CrewAI equivalents:

```
from crewai import Agent, Task, Crew

python_coder = Agent(
    role="Python Developer",
    goal="Write clean, tested Python code following standards",
    backstory=load_agent("python-coder"),
    tools=[WriteFileTool(), EditFileTool(), ReadFileTool()],
)

task = Task(
    description="Implement user authentication with FastAPI",
    agent=python_coder,
    expected_output="Working auth endpoints with tests"
)
```

```
crew = Crew(agents=[python_coder], tasks=[task])
```

AutoGen

Load agent markdown as system_message:

```
from autogen import AssistantAgent

python_coder = AssistantAgent(
    name="python_coder",
    system_message=load_agent("python-coder"),
    llm_config={"model": "gpt-4", "functions": [write_file, edit_file]}
)
```

IDE-Based Systems (Cursor, Windsurf, Aider, Continue)

These are typically single-agent systems. Copy the agent definition to the IDE's context file:

```
# Cursor
cp context/agents/python-coder.md .cursor/rules/python-coder.md

# Windsurf
cp context/agents/python-coder.md .windsurfrules

# Aider (load as additional context)
aider --read context/agents/python-coder.md \
      --read context/standards/tech-standards.md
```

Tool Mapping Reference

Claude Code Tool	LangGraph	CrewAI	AutoGen	IDE-Based
Write	write_file()	FileWriterTool	write_file() func	Built-in
Edit	edit_file()	FileWriterTool	edit_file() func	Built-in
Read	read_file()	FileReadTool	read_file() func	Built-in
Glob	glob.glob()	DirectorySearchTool	glob() func	Built-in
Grep	grep_content()	FileSearchTool	grep() func	Built-in
Bash	subprocess.run()	ShellTool	execute_code()	Built-in

Handling Frontmatter

Most frameworks do not parse YAML frontmatter automatically. Two approaches:

Strip frontmatter for frameworks that only need the system prompt:

```
def load_prompt_only(name: str) -> str:
    content = Path(f"context/agents/{name}.md").read_text()
    parts = content.split("---")
    return "---".join(parts[2:]) if len(parts) > 2 else content
```

Parse frontmatter for frameworks that can use the metadata (model choice, tool list, rule references):

```
import yaml

def load_with_metadata(name: str):
    content = Path(f"context/agents/{name}.md").read_text()
    parts = content.split("---")
    metadata = yaml.safe_load(parts[1]) if len(parts) > 2 else {}
    prompt = "---".join(parts[2:]) if len(parts) > 2 else content
    return {"prompt": prompt, "metadata": metadata}
```

How the Parts Connect

End to end: workflows pick which agent runs in each phase and which outputs must exist; the orchestrator loads that agent's rules (injected) and standards (referenced), wraps the task in the task prompt template, and dispatches execution; agents write artefacts that match templates; validators and hooks ensure the committed tree still matches the declared rules; state recovery blocks in YAML plus `HANDOFF.md` let a new session resume after compaction. The subsections below spell out each pairwise relationship.

Agents Read Rules and Standards

Agent files point to the rules and standards they depend on via frontmatter. The orchestrator resolves these references at spawn time, injecting rules and pointing the agent to the relevant standards. This keeps prompts small and composable: the agent definition is the single source of truth for what each specialist needs.

Workflows Sequence Agents

Workflow YAML does not replace agent definitions; it arranges them. The design separates role definition (who does what and how) from phase sequencing (when they do it and in what order). This means the same agent can appear in multiple workflows with different sequencing.

Templates Shape Outputs

Templates are chosen by the orchestrator and used by producing agents. Output conventions are an explicit part of orchestration rather than an afterthought. When an agent produces a review, a handoff, or a set of requirements, the template defines what “complete” looks like.

Scripts Enforce the Model

Without validators, generators, and hooks, the framework would rely on memory and goodwill. The scripts directory turns expectations into checks: validators catch rule violations at commit time, generators keep derived files in sync with authoritative sources, and hooks record session telemetry automatically.

Recovery is Built In

Every workflow YAML includes a `state_recovery` section listing the files an orchestrator should read to reconstruct its position after context compaction. Every agent handoff writes durable state to disk. The `HANDOFF.md` file records the active phase before agents are spawned, not after they report back, ensuring the phase survives even if compaction occurs during execution.

Reading Paths

Extending the Framework

1. `context/README.md`
2. `context/agents/orchestrator.md`
3. `context/workflows/build.yaml` **and** `context/workflows/design.yaml`
4. `context/templates/task-prompt-template.md`
5. Relevant rules and standards

Adding a New Agent

1. `context/agents/README.md`
2. `context/templates/agent-template.md`
3. `context/agents/orchestrator.md`
4. The workflow where the new agent should appear

Porting the Framework to Another Runtime

1. `README.md`
2. `context/README.md`
3. `AGENTS.md`

Understanding Quality Enforcement

1. `context/rules/`
2. `context/workflows/`
3. `context/scripts/validators/`
4. `context/standards/testing-standards.md`
5. `context/standards/doc-standards.md`

Running a Default Workflow

1. `context/workflows/design.yaml` **and** `context/workflows/build.yaml` (and `deploy.yaml` when promoting to cloud)
2. `context/agents/orchestrator.md`

Evaluating Workflow Health

1. `context/workflows/continuous-improvement.yaml`
2. `context/agents/workflow-analyst.md`
3. `context/standards/workflow-standards.md`

Writing Human-Facing Content (Blogs, Papers, Guides)

1. `context/workflows/content.yaml`
 2. `context/rules/no-ai-slop.mdc`
-



Thank you

Get in touch: wlytle@plandek.com



Learn more at plandek.com

